GGEngine

Lua для геймдизайнеров

Практическое руководство по скриптам игрового поведения

Движение объектов • Свет • Ввод • Физика

Здесь собраны доступные команды движка, инструкции по подключению скриптов и готовые примеры. Для работы достаточно редактора и текстового редактора Lua-файлов.

С чего начать

1. Подключите скрипт из раздела 1 и запустите игру.
2. Для управления объектом используйте пример WASD в разделе 9.
3. Для эффекта освещения возьмите пример из раздела 7.
4. Для физического толчка используйте пример из раздела 8.

Документ описывает текущие возможности проекта. Параметры поведения меняются в Lua-файле; автоматический вывод пользовательских полей скрипта в инспектор не предусмотрен.

Версия документа: 7 ноября 2026 года. Lua 5.4.8.

Содержание

1. Подключение скрипта 3

2. GameObject и поиск компонентов 5

3. Transform - узел дерева трансформации 6

4. Vector3 8

5. Quaternion 9

6. Color 10

7. Освещение 11

8. Физика 12

9. Игровой ввод 14

10. Библиотеки Lua и отладка 16

11. Что пока недоступно 17

Как читать примеры

Полные примеры начинаются с `local M = {}` и заканчиваются `return M`: ими можно заменить содержимое одного Lua-файла. Короткие фрагменты вставляются внутрь соответствующего обработчика.

Обозначение «Чтение» означает, что значение можно посмотреть, но нельзя присвоить ему новое. «Чтение/запись» разрешает оба действия.

`nil` означает, что значение отсутствует. `boolean` - логическое значение `true` (да) или `false` (нет). `dt` - длительность кадра в секундах.

1. Подключение скрипта

1. В Content Browser создайте `Lua script`, например `res/Scripts/Player.lua`.

2. Выберите объект и нужный узел его дерева трансформации. Добавьте компонент `Lua Script`.

3. В поле `Script path` укажите путь к файлу, например `res/Scripts/Player.lua`.

4. Запустите игру. В режиме редактирования обработчики скрипта не выполняются.

Пути - строки UTF-8 с разделителем `/`, без `res://`. Загрузка проходит через `ResourceManager.ResolvePath`: если менеджеру задан каталог проекта, относительный путь разрешается от него; иначе - от рабочей директории процесса. Абсолютные пути также поддерживаются. `res` - папка ресурсов, поэтому префикс `res/` в примере нужен.

Файл должен вернуть таблицу. Обработчики `on_init` и `on_update` необязательны:

```
local M = {}

function M.on_init(self)
    -- Состояние хранится отдельно для каждого компонента скрипта.
    self.speed = 2.0
    self.elapsed = 0.0
end

function M.on_update(self, dt)
    -- dt - время кадра в секундах; скорость задаём в единицах в секунду.
    self.elapsed = self.elapsed + dt
    self.transform:move(self.speed * dt, 0, 0)
end

return M
```

`on_init(self)` вызывается при инициализации компонента; `on_update(self, dt)` - при его покадровом обновлении. При повторной инициализации создаётся новая таблица `self`. Она не сохраняется в сцену. Автоматических обработчиков `on_destroy`, `on_fixed_update` и столкновений пока нет.

Ошибка загрузки, `on_init` или `on_update` выводится с префиксом `[Lua]` и отключает дальнейшее обновление данного скрипта до повторной инициализации.

Что находится в self

Поле	Значение
`self.object`	Объект `GameObject`, которому принадлежит компонент.
`self.transform`	Узел, к которому привязан компонент скрипта. Если узел не выбран, используется корень объекта.
`self.input`	Игровой ввод `GameInput`; предоставляется только в сборке с `GG_GAME`.
Собственные поля	Любое состояние скрипта: таймеры, параметры, найденные компоненты.

Методы модуля доступны через таблицу `self`. Для собственного метода можно написать `function M:jump()` и вызвать `self:jump()`.

Все скрипты одного `ScriptServer` используют общее Lua-состояние и глобальные переменные. Объявляйте вспомогательные переменные через `local`, а состояние экземпляра храните в `self`.

Точка и двоеточие

```
-- Фабрики типов вызываются через точку.
local offset = Vector3.new(1, 0, 0)
local rotation = Quaternion.identity()

-- Методы экземпляров - через двоеточие, свойства - через точку.
self.transform:move(offset)
self.transform.rotation = rotation
```

`value:method(arg)` эквивалентно `value.method(value, arg)`.

2. GameObject и поиск компонентов

API	Описание
`object.transform`	Корневой `Transform`; само свойство только для чтения, свойства узла изменяемы.
`object:get_component(type)`	Первый подходящий компонент среди компонентов всего объекта; `nil`, если он не найден или имя неизвестно.

Для поиска используйте константы `ComponentType`:

Константа	Lua-тип
`ComponentType.Mesh`	`Mesh`
`ComponentType.GameCamera`	`GameCamera`
`ComponentType.DirectionalLight`	`DirectionalLight`
`ComponentType.PointLight`	`PointLight`
`ComponentType.LuaScript`	`LuaScript`
`ComponentType.BoxCollider`	`BoxCollider`
`ComponentType.DynamicRigidBody`	`DynamicRigidBody`
`ComponentType.StaticRigidBody`	`StaticRigidBody`
`ComponentType.EditorCamera`	`EditorCamera`, только при `GG_EDITOR`
`ComponentType.CameraMovement`	`CameraMovement`, только при `GG_EDITOR`

Обычное изменение констант и замена их метатаблицы запрещены. Старые строковые имена, например `"PointLight"`, также принимаются, но константы удобнее для написания скриптов.

```
local light = self.object:get_component(ComponentType.PointLight)
if light then
    -- Поиск может вернуть nil: проверяем его перед обращением к свойствам.
    light.intensity = 3.0
    light.color = Color.new(1.0, 0.7, 0.3)
end
```

Поиск не ограничивается узлом `self.transform`. Если одинаковых компонентов несколько, выбрать конкретный по имени или получить список через Lua пока нельзя. `Transform:get_component` отсутствует.

Объекты, узлы и компоненты принадлежат C++. Lua получает ссылки на них; создавать и удалять их через этот API нельзя. Не используйте сохранённые ссылки после уничтожения соответствующего объекта или узла.

3. Transform - узел дерева трансформации

`self.transform` может быть веткой, а `self.object.transform` всегда обозначает корень объекта.

Свойства

Свойство	Тип	Доступ	Значение
`position`	`Vector3`	Чтение/запись	Локальная позиция.
`rotation`	`Quaternion`	Чтение/запись	Локальное вращение.
`scale`	`Vector3`	Чтение/запись	Локальный масштаб.
`inherit_position`	`boolean`	Чтение/запись	Наследование позиции родителя.
`inherit_rotation`	`boolean`	Чтение/запись	Наследование вращения родителя.
`inherit_scale`	`boolean`	Чтение/запись	Наследование масштаба родителя.
`world_position`	`Vector3`	Чтение	Мировая позиция.
`world_rotation`	`Quaternion`	Чтение	Мировое вращение.
`world_scale`	`Vector3`	Чтение	Мировой масштаб.
`parent`	`Transform` или `nil`	Чтение	Родительский узел.
`child_count`	Целое число	Чтение	Количество непосредственных дочерних узлов.

Векторы и кватернионы свойств возвращаются копиями. Записывайте изменённое значение обратно:

```
local position = self.transform.position
position.y = 2.0
self.transform.position = position

-- Короткая запись для полной замены позиции.
self.transform.position = Vector3.new(0, 2, 0)
```

Запись `self.transform.position.y = 2` меняет только временную копию и не перемещает узел. Это же правило относится к `rotation`, `scale` и цвету света.

Мировые свойства вычисляются разложением мировой матрицы. Если разложение невозможно, возвращаются нейтральные значения: нулевая позиция, единичное вращение и масштаб `(1, 1, 1)`.

Методы

Вызов	Действие
`t:move(offset)`	Прибавить `Vector3` к локальной позиции.
`t:move(x, y, z)`	То же, отдельными числами.
`t:rotate(axis, radians)`	Добавить вращение вокруг оси `Vector3`; ось нормализуется. Нулевая ось игнорируется.
`t:resize(x, y, z)`	Умножить текущий масштаб покомпонентно.
`t:look_at(target)`	Вычислить вращение через текущую реализацию `CreateLookAt`, с осью вверх `(0, 1, 0)`.
`t:look_at(target, up)`	То же с явно заданным вектором вверх.
`t:get_child(index)`	Получить непосредственного ребёнка по индексу; вне диапазона возвращает `nil`.

`move` не поворачивает смещение по ориентации узла: это прибавление к `position`. Для движения вдоль повёрнутого направления используйте `rotation:rotate(vector)`.

В `look_at` цель должна быть в той же системе координат, что и локальная `position`. Текущая реализация напрямую преобразует матрицу вида в кватернион; не следует считать её универсальным мировым наведением объекта. Совпадение цели с позицией и нулевой `up` игнорируются; избегайте также направления, параллельного `up`.

```
-- В отличие от обычных Lua-массивов, индексы узлов начинаются с 0.
for i = 0, self.transform.child_count - 1 do
    local child = self.transform:get_child(i)
    print(i, child.position)
end
```

Создание, удаление, переименование, переподчинение и поиск узлов по имени из Lua пока недоступны.

4. Vector3

Создание и поля

Вызов	Результат
`Vector3.new()`	Нулевой вектор.
`Vector3.new(x, y, z)`	Вектор с указанными компонентами.
`Vector3.zero()`	`(0, 0, 0)`
`Vector3.one()`	`(1, 1, 1)`
`Vector3.unit_x()`	`(1, 0, 0)`
`Vector3.unit_y()`	`(0, 1, 0)`
`Vector3.unit_z()`	`(0, 0, 1)`

Поля `v.x`, `v.y`, `v.z` доступны для чтения и записи. Фабрики возвращают новые значения.

Методы и операции

Вызов	Результат
`v:copy()`	Копия вектора.
`v:length()`	Длина.
`v:length_squared()`	Квадрат длины.
`v:normalized()`	Нормализованная копия; перед вызовом проверяйте, что длина ненулевая.
`a:dot(b)`	Скалярное произведение, число.
`a:cross(b)`	Векторное произведение.
`a:distance(b)`	Расстояние между векторами как точками.
`a:lerp(b, t)`	Линейная интерполяция; `t = 0` даёт `a`, `t = 1` - `b`.

Поддерживаются `a + b`, `a - b`, `-a`, `a * b`, `a / b`, `a * number`, `number * a`, `a / number`, `a == b`, `tostring(a)`. Умножение и деление двух векторов покомпонентные. Результаты арифметики - новые значения. Равенство точное, без допуска. Деление числа на вектор не привязано.

```
local direction = Vector3.new(1, 0, 1)
if direction:length_squared() > 0 then
    -- Нормализация сохраняет одинаковую скорость движения по диагонали.
    direction = direction:normalized()
    self.transform:move(direction * self.speed * dt)
end
```

5. Quaternion

Углы задаются в радианах. Для перевода градусов используйте `math.rad(degrees)`.

Вызов	Результат
`Quaternion.new()`	Единичный кватернион `(0, 0, 0, 1)`.
`Quaternion.new(x, y, z, w)`	Кватернион из четырёх компонент.
`Quaternion.identity()`	Единичное вращение.
`Quaternion.from_axis_angle(axis, radians)`	Вращение вокруг оси; ось нормализуется, для нулевой оси возвращается единичное вращение.
`Quaternion.from_euler(yaw, pitch, roll)`	Вращение из углов вокруг Y, X, Z соответственно, в порядке параметров yaw/pitch/roll.
`q:copy()`	Копия.
`q:normalized()`	Нормализованная копия.
`q:conjugate()`	Сопряжённый кватернион.
`q:rotate(v)`	Повернуть `Vector3` и вернуть новый вектор.
`a:slerp(b, t)`	Сферическая интерполяция между вращениями.

Поля `x`, `y`, `z`, `w` доступны для чтения и записи. Поддерживаются `a * b`, `a == b`, `tostring(q)`. В используемом SimpleMath произведение `a * b` применяет сначала вращение `a`, затем `b`. Для корректных вращений используйте нормализованные кватернионы.

```
-- Поворот на 90 градусов вокруг Y и преобразование направления.
local q = Quaternion.from_axis_angle(Vector3.unit_y(), math.rad(90))
self.transform.rotation = q
local direction = q:rotate(Vector3.unit_z())
```

6. Color

Вызов	Результат
`Color.new()`	Чёрный цвет с альфой 1.
`Color.new(r, g, b)`	Цвет с альфой 1.
`Color.new(r, g, b, a)`	Цвет с явно заданной альфой.
`color:copy()`	Копия.
`tostring(color)`	Строковое представление.

Поля `r`, `g`, `b`, `a` доступны для чтения и записи. Обычный диапазон компонент - от 0 до 1; привязка не ограничивает значения автоматически. Арифметические операции для `Color` не привязаны.

7. Освещение

`DirectionalLight` и `PointLight` наследуют свойства Lua-типа `Light`:

Свойство	Тип	Доступ	Назначение
`color`	`Color`	Чтение/запись	RGB света; альфа игнорируется, при чтении равна 1.
`intensity`	Число	Чтение/запись	Интенсивность.
`enabled`	`boolean`	Чтение/запись	Включение света.
`range`	Число	Чтение/запись	Дальность, только для `PointLight`.

Полный пример пульсирующего источника света:

```
local M = {}

function M.on_init(self)
    self.light = self.object:get_component(ComponentType.PointLight)
    self.elapsed = 0
end

function M.on_update(self, dt)
    if not self.light then return end

    -- Меняем интенсивность между 1 и 3 с периодом 2 секунды.
    self.elapsed = self.elapsed + dt
    self.light.intensity = 2 + math.sin(self.elapsed * math.pi)
end

return M
```

8. Физика

DynamicRigidBody

Метод	Действие
`body:add_force(Vector3)`	Добавить силу на ближайший физический шаг.
`body:add_impulse(Vector3)`	Добавить линейный импульс.
`body:add_torque(Vector3)`	Добавить момент силы на ближайший физический шаг.
`body:add_angular_impulse(Vector3)`	Добавить угловой импульс.

Все векторы задаются в мировых координатах. Методы возвращают `boolean`: команда принята в очередь или отклонена. `false` возвращается при неправильных аргументах, выключенном теле, неконечных значениях или переполнении накопленной команды. `true` ещё не означает, что тело физически переместилось: например, без созданного физического тела команды будут сброшены на шаге.

Команды суммируются до ближайшего fixed-step, затем очищаются. Сила и момент действуют один шаг; импульсы применяются однократно. Если между физическими шагами вызвать `add_force` несколько раз, силы сложатся. Если в кадре несколько физических шагов, команда не повторяется автоматически на каждом из них. Поэтому вызов силы из `on_update` сам по себе не обеспечивает постоянную силу независимо от частоты кадров. Lua-обработчика fixed-step пока нет.

Свойство	Тип	Доступ	Значение
`linear_velocity`	`Vector3`	Чтение	Линейная скорость.
`angular_velocity`	`Vector3`	Чтение	Угловая скорость.
`mass`	Число	Чтение	Масса тела.
`has_actor`	`boolean`	Чтение	Создано ли физическое тело.
`is_sleeping`	`boolean`	Чтение	Спит ли тело.
`enabled`	`boolean`	Чтение/запись	Включено ли тело.

Скорости возвращаются копиями и отражают состояние после завершённого физического шага. Прямой setter скорости и массы из Lua отсутствует. Создание физического тела происходит при обработке физикой, поэтому в `on_init` `has_actor` может быть ещё `false`.

StaticRigidBody и BoxCollider

Тип	Свойство	Доступ	Значение
`StaticRigidBody`	`has_actor`	Чтение	Наличие физического тела.
`StaticRigidBody`	`enabled`	Чтение/запись	Включение тела.
`BoxCollider`	`size`	Чтение	Размер как копия `Vector3`.
`BoxCollider`	`enabled`	Чтение/запись	Включение коллайдера.

У статического тела нет методов сил и импульсов. Изменение `enabled` учитывается физической системой на шаге. Для динамического тела настройте корневой узел и коллайдер в редакторе; Lua не создаёт их автоматически.

Пример одиночного толчка по пробелу (не проверяет нахождение на земле):

```
local M = {}

function M.on_init(self)
    self.body = self.object:get_component(ComponentType.DynamicRigidBody)
end

function M.on_update(self, dt)
    if not self.input or not self.input.active then return end
    if not self.body or not self.body.has_actor then return end

    -- Импульс по нажатию применяется один раз, умножать его на dt не нужно.
    if self.input:was_key_pressed(Keys.Space) then
        local accepted = self.body:add_impulse(Vector3.new(0, 5, 0))
        if not accepted then print("Импульс отклонён") end
    end
end

return M
```

9. Игровой ввод

`self.input`, тип `GameInput` и таблица `Keys` доступны в игровой сборке (`GG_GAME`).

Метод	Когда использовать
input:is_key_down(key)	Непрерывное движение, пока кнопка удерживается.
input:was_key_pressed(key)	Одно действие при нажатии: толчок, переключение света.
input:was_key_released(key)	Действие при отпускании кнопки.

Свойства только для чтения

Свойство	Значение
active	Ввод включён, игровое окно имеет фокус.
mouse_x, mouse_y	Позиция мыши из системы ввода.
mouse_delta_x, mouse_delta_y	Суммарное смещение мыши за кадр.
wheel_delta	Суммарная прокрутка за кадр.

Несколько скриптов могут прочитать одно нажатие. Удержание клавиши не создаёт новых событий was_key_pressed. При потере фокуса состояние очищается.

Дельты мыши и колеса сбрасываются каждый кадр. Это уже смещения: умножать их на dt не нужно.

Доступные клавиши: `Keys.A` ... `Keys.Z`, `Keys.D0` ... `Keys.D9`, `Keys.Escape`, `Keys.Space`, `Keys.LeftShift`, `Keys.RightShift`, `Keys.Up`, `Keys.Down`, `Keys.Left`, `Keys.Right`.

Кнопки мыши: `Keys.LeftButton`, `Keys.RightButton`, `Keys.MiddleButton`. Имена `Keys.Enter`, `Keys.Control` и `Keys.F1` не зарегистрированы.

Практика: движение по WASD

Подключите скрипт к узлу объекта без динамического физического управления. Запустите игру и установите фокус на игровом окне.

W/S меняют координату Z, A/D - координату X. Направление задаётся относительно родителя узла, а не относительно камеры.

```
local M = {}

function M.on_init(self)
    self.speed = 4.0
end

function M.on_update(self, dt)
    local input = self.input
    if not input or not input.active then return end

    -- Направление движения в плоскости XZ.
    local direction = Vector3.zero()
    if input:is_key_down(Keys.W) then direction.z = direction.z + 1 end
    if input:is_key_down(Keys.S) then direction.z = direction.z - 1 end
    if input:is_key_down(Keys.D) then direction.x = direction.x + 1 end
    if input:is_key_down(Keys.A) then direction.x = direction.x - 1 end

    -- Одинаковая скорость по прямой и диагонали.
    if direction:length_squared() > 0 then
        self.transform:move(direction:normalized() * self.speed * dt)
    end
end

return M
```

Что настроить

Параметр	Результат
self.speed = 4.0	Скорость: 4 единицы за секунду. Увеличьте число, чтобы ускорить объект.
Keys.W / S / A / D	Замените константы, чтобы назначить другие кнопки.

Для физического персонажа используйте команды тела из раздела 8. Пример выше непосредственно изменяет позицию узла.

10. Библиотеки Lua и отладка

Открыты базовая библиотека, `math`, `string`, `table`, `utf8`. Например, доступны `pairs`, `ipairs`, `type`, `tostring`, `math.sin`, `math.rad`, `string.format`, `table.insert`.

`io`, `os`, `package`, `debug`, `coroutine` не открываются. Стандартный `require` недоступен, поскольку `package` не подключён. Это перечень открытых библиотек, а не гарантия изоляции: базовая библиотека Lua также содержит функции загрузки кода.

`print(...)` выводит аргументы через табуляцию с префиксом `[Lua]`; для математических типов используется их строковое представление:

```
print("Позиция:", self.transform.position)
print("Компонент найден:", self.object:get_component(ComponentType.Mesh) ~= nil)
```

При диагностике проверяйте:

1. Файл возвращает таблицу через `return M`.
2. Обработчики названы именно `on_init` и `on_update`.
3. Игра запущена: в режиме редактирования скрипты не обновляются.
4. Методы экземпляров вызываются через `:`, свойства читаются через `.`.
5. Результат `get_component` проверяется на `nil`.
6. Изменённые векторные свойства присваиваются целиком.
7. Углы заданы в радианах, а индексы дочерних узлов начинаются с нуля.
8. Для физики есть действующее тело и коллайдер; сообщение об ошибке ищется в выводе `[Lua]`.

11. Что пока недоступно

• Создание и удаление объектов, компонентов и узлов из Lua.

• Доступ к уровню, поиск других объектов, загрузка ресурсов и переключение сцен через Lua API.

• Поиск веток по именам и поиск компонента только на выбранном узле.

• Изменение мешей, материалов и параметров камеры.

• Настройка размера коллайдера, массы и скорости тела из Lua.

• Обработчики столкновений, fixed-step, завершения скрипта и автоматическая горячая перезагрузка.

`Mesh`, `GameCamera`, `LuaScript`, а также редакторские `EditorCamera` и `CameraMovement` зарегистрированы как типы без пользовательских методов и свойств. Их можно найти и проверить на наличие. Наличие метода в C++ не означает, что он доступен в Lua.

О документе

Справочник подготовлен по LUA_API.md и текущим Lua-привязкам проекта: ScriptBindingsMath.cpp, ScriptBindingsTransform.cpp, ScriptBindingsComponents.cpp, LuaScriptComponent.cpp и ScriptServer.cpp. Примеры сверены с доступным API; в игровом режиме в рамках подготовки PDF не запускались.